

Jetpack Compose 基础教程

Jetpack Compose第一章第一节教学讲义（零基础入门）

各位同学，今天咱们正式开启**Jetpack Compose**的学习之旅！它是Google官方推出的**新一代Android UI开发框架**，用“声明式编程”彻底颠覆了传统的XML布局方式。这节课从“是什么、为什么学”讲到“核心用法”，全程代码+通俗解释，保证零基础也能轻松上手！

一、Jetpack Compose是什么？有什么优势？

1. 它是什么？

简单说，Jetpack Compose是一种**用代码直接写UI**的技术，不再需要XML文件。你可以把它理解为“用Kotlin代码画界面”，比如按钮、文本、列表这些UI元素，都可以通过Kotlin函数直接创建。

2. 相比传统XML，它的核心优势

传统XML方式	Jetpack Compose方式
需维护XML和Kotlin两套代码	纯Kotlin代码，逻辑与UI高度内聚
容易出现空指针（如控件未找到）	编译期检查，几乎杜绝UI相关空指针
动态UI需写大量 findViewById	状态驱动UI，数据变了UI自动刷新
自定义View逻辑复杂	组合式API，自定义UI像搭积木一样简单

举个直观对比：

Code block

```
1  <!-- 传统XML写一个按钮 -->
2  <Button
3      android:id="@+id/btn"
4      android:layout_width="wrap_content"
5      android:layout_height="wrap_content"
6      android:text="点击我" />
7
8  // 传统Kotlin处理点击
9  btn.setOnClickListener {
10      Toast.makeText(this, "点击了按钮", Toast.LENGTH_SHORT).show()
11  }
```

Code block

```
1  // Compose写一个按钮（逻辑与UI内聚）
2  @Composable
3  fun MyButton() {
4      Button(onClick = {
5          Toast.makeText(LocalContext.current, "点击了按钮",
6              Toast.LENGTH_SHORT).show()
7      }) {
8          Text("点击我")
9      }
10 }
```

二、Composable函数：Jetpack Compose的“UI积木”

1. 什么是Composable函数？

用 `@Composable` 注解标记的函数，就是“可组合函数”——它是构成Compose UI的基本单元，负责生成一个UI元素（比如按钮、文本、整个页面）。

核心特点：

- 无返回值，直接生成UI；
- 可嵌套调用（像搭积木一样组合UI）；
- 由状态驱动（状态变了，UI自动刷新）。

2. 第一个Composable函数：Hello World

Code block

```
1  import androidx.compose.material3.Text
2  import androidx.compose.runtime.Composable
3
4  // 定义一个可组合函数：显示Hello Compose文本
5  @Composable
6  fun HelloCompose() {
7      Text(text = "Hello Jetpack Compose!")
8  }
```

3. 预览函数：不用运行APP就能看UI

用 `@Preview` 注解标记的函数，可在Android Studio的“Preview”面板直接预览UI，无需编译运行。

Code block

```
1  import androidx.compose.ui.tooling.preview.Preview
2
3  // 预览函数：预览HelloCompose的效果
4  @Preview(showBackground = true) // showBackground显示背景
5  @Composable
6  fun HelloComposePreview() {
7      HelloCompose()
8  }
```

此时在Android Studio的Preview面板，就能看到“Hello Jetpack Compose!”的文本预览了。

三、布局：Compose的“容器积木”

Compose提供了多种**布局组件**，用于控制UI元素的排列方式（如垂直排列、水平排列、居中、占满全屏等）。

1. Column：垂直排列元素

像“垂直的LinearLayout”，子元素从上到下排列。

Code block

```
1  import androidx.compose.foundation.layout.Column
2  import androidx.compose.material3.Text
3
4  @Composable
5  fun ColumnExample() {
6      Column {
7          Text("第一行文本")
8          Text("第二行文本")
9          Text("第三行文本")
10     }
11 }
12
13 @Preview(showBackground = true)
14 @Composable
15 fun ColumnExamplePreview() {
16     ColumnExample()
17 }
```

2. Row：水平排列元素

像“水平的LinearLayout”，子元素从左到右排列。

Code block

```
1  import androidx.compose.foundation.layout.Row
2
3  @Composable
4  fun RowExample() {
5      Row {
6          Text("左")
7          Text("中")
8          Text("右")
9      }
10 }
11
12 @Preview(showBackground = true)
13 @Composable
14 fun RowExamplePreview() {
15     RowExample()
16 }
```

3. Box：层叠或对齐元素

像“FrameLayout”，子元素可以层叠，或在容器内对齐（如居中、居左等）。

Code block

```
1  import androidx.compose.foundation.layout.Box
2  import androidx.compose.foundation.layout.fillMaxSize
3  import androidx.compose.ui.Alignment
4
5  @Composable
6  fun BoxExample() {
7      Box(
8          modifier = Modifier.fillMaxSize(), // 占满父容器
9          contentAlignment = Alignment.Center // 子元素居中
10     ) {
11         Text("居中的文本")
12     }
13 }
14
15 @Preview(showBackground = true)
```

```
16 @Composable
17 fun BoxExamplePreview() {
18     BoxExample()
19 }
```

四、配置布局：用Modifier精细控制UI

`Modifier` 是Compose中控制UI大小、边距、背景、点击事件等属性的“配置器”，可以链式调用多个配置。

1. 常用Modifier配置

Code block

```
1  import androidx.compose.foundation.background
2  import androidx.compose.foundation.clickable
3  import androidx.compose.foundation.layout.*
4  import androidx.compose.ui.graphics.Color
5  import androidx.compose.ui.unit.dp
6
7  @Composable
8  fun ModifierExample() {
9      Box(
10         modifier = Modifier
11             .fillMaxWidth() // 占满父容器宽度
12             .height(100.dp) // 高度100dp
13             .padding(16.dp) // 内边距16dp
14             .background(Color.Blue) // 背景蓝色
15             .clickable { // 点击事件
16                 Toast.makeText(LocalContext.current, "点击了蓝色区域",
17                     Toast.LENGTH_SHORT).show()
18             }
19         ) {
20             Text("带Modifier的Box", color = Color.White)
21         }
22
23     @Preview(showBackground = true)
24     @Composable
25     fun ModifierExamplePreview() {
```

```
26     ModifierExample()
27 }
```

五、Material Design：Compose的“风格套装”

Compose内置了**Material Design 3**组件（如按钮、卡片、顶部栏等），直接调用即可实现符合Material风格的UI。

1. 按钮（Button）

Code block

```
1  import androidx.compose.material3.Button
2
3  @Composable
4  fun MaterialButton() {
5      Button(
6          onClick = { /* 点击逻辑 */ },
7          modifier = Modifier.padding(8.dp)
8      ) {
9          Text("Material按钮")
10     }
11 }
12
13 @Preview(showBackground = true)
14 @Composable
15 fun MaterialButtonPreview() {
16     MaterialButton()
17 }
```

2. 卡片（Card）

Code block

```
1  import androidx.compose.material3.Card
```

```

2  import androidx.compose.material3.Text
3
4  @Composable
5  fun MaterialCard() {
6      Card(
7          modifier = Modifier
8              .width(200.dp)
9              .padding(8.dp),
10         elevation = CardDefaults.cardElevation(4.dp) // 阴影
11     ) {
12         Column(modifier = Modifier.padding(16.dp)) {
13             Text("卡片标题")
14             Text("卡片内容, Material风格")
15         }
16     }
17 }
18
19 @Preview(showBackground = true)
20 @Composable
21 fun MaterialCardPreview() {
22     MaterialCard()
23 }

```

六、列表与状态：动态数据的“展示与驱动”

1. 列表（LazyColumn/LazyRow）

用于展示“大量或动态”的列表数据，只渲染可见区域，性能高效（类似RecyclerView）。

Code block

```

1  import androidx.compose.foundation.lazy.LazyColumn
2  import androidx.compose.foundation.lazy.items
3  import androidx.compose.material3.Text
4
5  @Composable
6  fun UserList(users: List<String>) {
7      LazyColumn {
8          // 固定项
9          item {

```



```

10         Text("用户列表", modifier = Modifier.padding(16.dp))
11     }
12     // 遍历数据项
13     items(users) { user ->
14         Text("用户名: $user", modifier = Modifier.padding(8.dp))
15     }
16 }
17 }
18
19 @Preview(showBackground = true)
20 @Composable
21 fun UserListPreview() {
22     UserList(listOf("小明", "小红", "小刚", "小李"))
23 }

```

2. 状态 (State)：驱动UI刷新

Compose是“状态驱动UI”，用 `mutableStateOf` 定义的状态变量，值变化时UI会自动刷新。

Code block

```

1  import androidx.compose.runtime.mutableStateOf
2  import androidx.compose.runtime.remember
3  import androidx.compose.material3.Button
4  import androidx.compose.material3.Text
5
6  @Composable
7  fun Counter() {
8      // 定义状态: 计数
9      val count = remember { mutableStateOf(0) }
10
11      Column {
12          Text("计数: ${count.value}")
13          Button(onClick = { count.value++ }) {
14              Text("增加")
15          }
16      }
17  }
18
19 @Preview(showBackground = true)
20 @Composable
21 fun CounterPreview() {

```

```
22     Counter()
23 }
```

点击“增加”按钮时，`count.value` 变化，Text会自动刷新显示新的计数。

总结与作业

核心知识点

- Jetpack Compose是用Kotlin代码写UI的新一代框架，优势是“逻辑内聚、编译期安全、状态驱动”；
- `@Composable` 函数是UI的基本单元，`@Preview` 函数用于预览；
- 布局组件（Column、Row、Box）用于排列UI，Modifier用于精细配置；
- Material Design组件（Button、Card等）可直接调用，实现Material风格；
- 列表用LazyColumn/LazyRow，状态用mutableStateOf驱动UI刷新。

课后作业

1. 写一个Composable函数，包含Column布局，内部有一个Material按钮和一个文本；
2. 给按钮添加点击事件，点击后文本内容从“未点击”变成“已点击”（用状态实现）；
3. 用@Preview预览效果。

试着自己写代码实现，有问题随时问哦！

（注：文档部分内容可能由 AI 生成）